
This is a copy of a shared ChatGPT conversation

Report conversation

Show more ▾

Yes — the timing numbers are only one slice of the value. For this kind of MVP/portfolio app, Instruments is more useful as a **sanity check for regressions and obvious resource problems** than as "search took 412 ms."

You're right to question it.

For your app, I'd want Instruments to answer at least these questions:

1. Is anything obviously slow?

The ms numbers help here, but only if there's a baseline and repeated runs.

2. Does memory grow and never come back down?

Yes, check this. Run:

- **Leaks** instrument
- **Allocations** instrument
- possibly **Memory Graph** in Xcode after repeated flows

3. Are views / view models being retained accidentally?

Especially after:

- Search → detail → back
- Add/remove wishlist repeatedly
- Empty search / failed network path
- Theme changes, if applicable

4. Is the main thread blocked?

Use **Time Profiler / Hangs** to see whether JSON parsing, image loading, persistence, or filtering blocks UI.

5. Are network calls sane?

For Search and Details:

- no duplicate requests
- failures handled cleanly
- cancellation/back navigation doesn't leave weird work running

So I'd treat Cursor's report as incomplete, not useless. Timing should be a small section, something like:

"All tested flows completed quickly with no user-visible delay. No meaningful performance concern found."

But the stronger report would add:

"No leaks observed after repeated navigation and wishlist mutation flows."

"Memory returned near baseline after repeated search/detail cycles."

"No main-thread stalls observed during search, detail loading, or persistence updates."

"No excessive duplicate network calls observed."

For a portfolio project, that is much more valuable than raw milliseconds. It shows you understand what performance profiling is actually for.

What should I tell it to get a useful dataset?

Tell Cursor to produce a **profiling report**, not just timings:

Please rerun Instruments and report a useful performance dataset, not only elapsed times. 

For each flow:

- Cold launch
- Cold launch with existing data
- Search
- Search failure / empty results
- Show Details
- View Wishlist
- Add to Wishlist
- Remove from Wishlist

Collect:

1. Wall-clock duration, averaged over at least 5 runs
2. Peak memory
3. Memory after the flow returns to idle
4. Whether memory returns near baseline
5. Leaks detected by Instruments Leaks
6. Retained view/view model instances after navigation back
7. Main-thread hangs or long blocks over 100 ms
8. Top CPU-heavy methods from Time Profiler
9. Network requests made, including duplicate/unexpected requests
10. Any console errors, warnings, or Swift concurrency/runtime warnings

Also run stress loops:

- Search → Details → Back, 20 times
- Add/remove wishlist item, 50 times
- Failed/empty search, 20 times
- Launch with existing wishlist data, 10 times

Please summarize:

- Pass/fail for each flow
- Any suspected leaks or retain cycles
- Any main-thread work that should move off-main
- Any suspicious memory growth
- Whether the app is safe for beta from a performance/resource perspective
- Raw Instruments artifacts saved, but do not commit .trace or .instruments files.

The important shift is: **"Does the app stay healthy after repeated use?"**, not **"How many ms did one happy-path action take?"**

That's good. Also, I've noticed an occasional crash when the app is sitting idle. It can take hours to happen. How can I best track this down?

For an **hours-later idle crash**, ask Cursor to make the app collect evidence first.

Please help track an intermittent idle crash that can happen hours after launch. 

Add/verify:

1. Xcode Organizer crash log collection for TestFlight/device builds
2. OSLog logging around:
 - app launch
 - scene phase changes
 - background/foreground transitions
 - wishlist persistence reads/writes
 - search/detail network requests
 - timer/task start and cancellation
 - any cache cleanup or refresh logic
3. Ensure every Task is cancellable and does not strongly retain view models unnecessarily.
4. Check for force unwraps, try!, fatalError, preconditionFailure, array index assumptions, a
5. Add breadcrumbs so crash logs show the last meaningful app activity.
6. Run a long idle test on-device:
 - launch app
 - optionally populate wishlist
 - leave on Search tab for 6+ hours
 - repeat on Wishlist tab
 - repeat after backgrounding/foregrounding
7. During long idle test, collect:
 - crash log
 - device console logs
 - memory graph if memory rises
 - Instruments Allocations/Leaks if practical

Also tell it:

Do not guess the cause. First identify the crash signature: exception type, crashed thread, t 

Most useful evidence will be the **symbolicated crash log** from Xcode Organizer or the device. For an idle crash, raw Instruments timing is much less useful than: “what thread crashed, what stack frame, and what was the app doing right before that?”